

Reviewer Pack — Minimal Symplectic Volume of Affine Varieties and Resolution...

Entry b6a5c0845f16 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 02:05:30 UTC

Minimal Symplectic Volume of Affine Varieties and Resolution Proof Depth

Entry ID: b6a5c0845f16 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 02:05:30 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

[‘For each affine variety V with resolution proof depth $d(V) \leq n$, the minimal symplectic volume of V is at most $2^{d(V)}$.’, ‘This implies that for a Tseitin formula G , if its associated variety has minimal symplectic volume at least $2^{n/2}$, then G requires a Resolution refutation of length at least 2^n .’, ‘For all other affine varieties with minimal symplectic volume less than $2^{n/2}$, the corresponding Tseitin formulas can be resolved in linear depth.’]

Rationale (proposer’s reasoning):

[‘Symplectic geometry provides geometric invariants that could capture nontrivial structural properties of computational problems.’, ‘Minimal symplectic volume might be a novel invariant that correlates with resolution proof complexity for Tseitin formulas, offering new insights into the hardness of propositional logic.’, ‘This connection has not been explored before and could potentially expose new techniques or barriers in complexity theory.’]

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bf51594478a1b995

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjectural result is supported if the minimal symplectic volume of an affine variety V with $d(V) \leq n$ is at most $2^{d(V)}$, where all results are derived from a statistically significant sample of Tseitin formulas G on n variables, using a geometric algorithm with polynomial complexity in n and known upper bounds for resolution proof depth.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - (symplectic geometry AND resolution proof complexity) AND (affine variety OR Tseitin formula) - (minimal symplectic volume AND resolution proof depth) AND (linear depth OR $2^d(V)$) - (resolution refutation length AND $2^{n/2}$) AND (affine variety OR Tseitin formula)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
```

```

        for k in range(n):
            C[i][j] += A[i][k] * B[k][j]
    return C

def symplectic_volume(A):
    det = 1
    n = len(A)
    for i in range(n):
        det *= A[i][i]
    return abs(det)

def resolution_depth(G):
    # Placeholder function to simulate resolution depth calculation
    # This is a dummy implementation and should be replaced with actual logic
    return random.randint(1, 20)

n = random.choice([5, 10, 15, 20, 30, 40])
G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
V = gaussian_elimination(G)
volume = symplectic_volume(V)
d_V = resolution_depth(G)

metric_name = "minimal_symplectic_volume"
metric_value = volume
instances_tested = 1
conjecture_holds = volume <= 2 ** d_V
counterexample = "" if conjecture_holds else f"Volume: {volume}, Depth: {d_V}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}'\n first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'metric_name': 'minimal_symplectic_volume', 'metric_value': 1.0, 'instances_tested': 1, 'conjecture': 'minimal_symplectic_volume'}
TRIAL: {'metric_name': 'minimal_symplectic_volume', 'metric_value': 1.0, 'instances_tested': 1, 'conjecture': 'minimal_symplectic_volume'}
TRIAL: {'metric_name': 'minimal_symplectic_volume', 'metric_value': 1.0, 'instances_tested': 1, 'conjecture': 'minimal_symplectic_volume'}
TRIAL: {'metric_name': 'minimal_symplectic_volume', 'metric_value': 1.0, 'instances_tested': 1, 'conjecture': 'minimal_symplectic_volume'}
TRIAL: {'metric_name': 'minimal_symplectic_volume', 'metric_value': 1.0, 'instances_tested': 1, 'conjecture': 'minimal_symplectic_volume'}
TRIAL: {'metric_name': 'minimal_symplectic_volume', 'metric_value': 1.0, 'instances_tested': 1, 'conjecture': 'minimal_symplectic_volume'}
TRIAL: {'metric_name': 'minimal_symplectic_volume', 'metric_value': 1.0, 'instances_tested': 1, 'conjecture': 'minimal_symplectic_volume'}
```

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe6c32fb.py", line 85, in <module>
    result = run_trial(seed)
             ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe6c32fb.py", line 62, in run_trial
    V = gaussian_elimination(G)
       ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fe6c32fb.py", line 31, in gaussian_elimination
    A[i][j] /= factor
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the required computations and verify the conjecture. | next: Investigate the cause of the crash in the test code and attempt to run the test again. If the crash is resolved, retest the conjecture with a larger sample size or under different conditions to ensure robustness.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14686
2	preregistration	ollama_remote	glm4:latest	0	0	9475
3	novelty	ollama_remote	glm4:latest	0	0	8747
4	novelty	ollama_remote	glm4:latest	0	0	8850
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13870
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12098
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9793

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9904
9	judge	ollama_remote	glm4:latest	0	0	12202

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 99624 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/b6a5c0845f16.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b6a5c0845f16.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b6a5c0845f16.tar.gz (if generated)